

	GUIA PARA LAS PRÁCTICAS DE LABORATORIO, TALLER O CAMPO	Departamento de Ciencias de la Computación
---	---	---

DEPARTAMENTO:	CIENCIAS DE LA COMPUTACIÓN	CARRERA:	INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN		
ASIGNATURA:	Programación Avanzada	PERÍODO LECTIVO:	202551	NIVEL:	7mo
DOCENTE:	Ing. Luis Castillo, Mgtr.	NRC:	29100	PRÁCTICA N°:	6
LABORATORIO DONDE SE DESARROLLARÁ LA PRÁCTICA		Laboratorio G-206			
TEMA DE LA PRÁCTICA:	Build, pruebas y CI				
INTRODUCCIÓN:					
En este laboratorio se integran conceptos fundamentales del desarrollo de software moderno, combinando el uso del control de versiones con Git y GitHub, la automatización del proceso de construcción mediante Gradle y la implementación de pruebas unitarias con JUnit 5 y Mockito. A través de esta práctica guiada, se aplicará un flujo de trabajo profesional basado en ramas, se validará la calidad del código mediante pruebas automatizadas y se utilizará integración continua para asegurar que los cambios realizados no afecten la estabilidad del proyecto. Este laboratorio permite comprender cómo las pruebas y la automatización forman parte esencial del ciclo de vida del software y del aseguramiento de la calidad en entornos reales de desarrollo.					
OBJETIVOS:					
● Aplicar un flujo de trabajo basado en control de versiones, utilizando ramas, commits y pull requests en GitHub, para gestionar de forma ordenada los cambios realizados sobre un proyecto de software. ● Automatizar la ejecución de pruebas unitarias y el proceso de construcción del proyecto, empleando Gradle y herramientas de integración continua, con el fin de validar la calidad y estabilidad del código antes de su integración. ● Desarrollar y verificar funcionalidades mediante pruebas unitarias, utilizando JUnit 5 y Mockito, asegurando que las reglas de negocio se cumplan y que las dependencias externas sean correctamente simuladas.					
MATERIALES:					
REACTIVOS: No aplica		INSUMOS: ● Una PC con Windows/Linux ● IntelliJ IDEA Ultimate ● Acceso a Internet			
EQUIPOS:					
Windows 10 o superior, Procesador Intel® Core™ i7-6700T o superior, 12GB RAM o superior, 480GB SSD o superior, Intel HD Graphics 530, similar o superior.					
MUESTRA: No aplica					
INSTRUCCIONES:					
1. Utilizar como material principal de apoyo, aquel indicado en clase por el docente. 2. No olvide incluir capturas de pantallas de todas las actividades realizadas durante la práctica. 3. En los datos ingresados, por favor usar sus datos personales, con el fin de verificar la realización de este trabajo. 4. Se debe comentar el código como mejor práctica de programación.					
ACTIVIDADES POR DESARROLLAR:					
PARTE 1: Creación del proyecto base					
Paso 1: Crear un nuevo proyecto de SpringBoot en IntelliJ					
a. Escoger bien la configuración del proyecto especialmente la versión de java.					

- b. No es necesario añadir dependencias ya que el proyecto que se crea ya tiene la dependencia *spring-boot-starter-test* (verificar que esté incluido en el archivo *build.gradle*).

Paso 2: Crear la estructura del proyecto.

- a. Crear las siguientes carpetas, dentro del src:
- src/main/java/...
 - service/
 - repository/
 - model/
 - dto/

PARTE 2: Implementación mínima**Paso 1: Crear Modelo.**

- a. Una cuenta tiene: id, ownerEmail, balance.
- b. Crear constructor lleno donde el id se llene de forma randómica
- c. Generar getters de todos los campos
- d. Generar dos funciones: una para depositar y otra para retirar dinero

Paso 2: Crear DTO de confirmación.

- a. En este DTO de confirmación se crearán dos funciones que permitan:
- Obtener el Id de la cuenta
 - Obtener el balance de la cuenta

PARTE 3: Dependencias a mockear**Paso 1: Implementar la dependencia WalletRepository.**

- a. Implementar una interfaz con tres métodos:
- Para guardar una cuenta
 - Para buscar una cuenta con un id específico
 - Para saber si existe una cuenta con el email del dueño

Paso 2: Implementar la dependencia RiskClient.

- a. Implementar una interfaz con un método para saber si un cliente ha sido bloqueado por riesgo.

Paso 4: Implementar el servicio WalletService.

- a. Crear objetos pertenecientes a las dependencias e inicializarlas desde el constructor.

- b. Implementar la función para crear una cuenta con la siguiente lógica:
 - email debe contener @
 - Saldo inicial debe ser mayor a 0
 - Rechazar al cliente si el cliente está “bloqueado”, llamar al servicio externo
 - Rechazar si ya existe una cuenta con ese email
 - Guardar en repositorio
 - Retornar un DTO con el id y el balance de la cuenta
- c. Implementar la función para realizar un depósito con la siguiente lógica:
 - El monto depositado debe ser mayor a 0
 - La cuenta debe existir
 - Usar la función de depositar del modelo
 - Persistir el saldo
 - Retornar el balance de la cuenta

PARTE 4: Pruebas unitarias (JUnit 5 + Mockito, AAA)

Paso 1: Crear la clase de prueba.

- a. Manejo del decorador @BeforeEach.
- b. Crear los siguientes casos de prueba:
 - Crear cuenta con datos válidos, guardar y retornar respuesta
 - Crear cuenta con correo no válido, lanzar excepción y no crear las dependencias
 - Depositar a una cuenta que no se encuentra y lanzar excepción
 - Depositar a una cuenta, se debe actualizar el balance, se debe guardar y usar captor

Paso 2: Integrar con Git.

- a. Inicializar git.
- b. Añadir todo el proyecto
- c. Realizar el commit para guardar los cambios

Paso 3: Ejecutar pruebas y build.

- a. Ejecutar pruebas `./gradlew test`.
- b. Ejecutar build `./gradlew build`

Paso 4: Subir a repositorio remoto.

- a. `git remote add origin https://github.com/TU_USUARIO/nombreRepositorio.git`
- b. `git push -u origin main`

Paso 5: Gestionar una nueva rama.

- a. Crear una nueva rama (feature)
- b. Añadir nueva funcionalidad

- Implementar en WalletService un método para realizar un retiro
 - i. amount > 0
 - ii. wallet debe existir
 - iii. saldo suficiente (si no, excepción)
 - iv. actualizar saldo y guardar
- c. Crear pruebas para esa funcionalidad
 - Withdraw exitoso
 - Withdraw fondos insuficientes
- d. Ejecutar tests y build en feature
- e. Commit de la feature

Paso 6: Subir la rama al repositorio remoto.

- a. git push -u origin feature
- b. git push -u origin main

Paso 7: Merge controlado a main.

- a. Cambiar a main y hacer merge:
 - git checkout main
 - git merge feature/withdraw
- b. Ejecutar nuevamente:
 - ./gradlew test
 - ./gradlew build

Paso 8: Generar CI con GitHub Actions.

- a. Crear el archivo `.github/workflows/ci.yml`
- b. El archivo de CI debe contener:
 - Nombre: CI
 - On: (se establece en clase)
 - Jobs: (se establecen en clase)

SECCIÓN DE PREGUNTAS/ACTIVIDADES**Actividad 1 — Integración remota mediante Pull Request y Merge en GitHub**

Se deberá realizar la integración de la funcionalidad desarrollada en la rama feature hacia la rama main utilizando GitHub, aplicando el mecanismo de Pull Request. Para ello, se deberá asegurar que las pruebas automatizadas se ejecuten correctamente antes de realizar el merge, garantizando que el código integrado no afecte la estabilidad del proyecto.

Instrucciones

1. Verificar que la rama feature se encuentre correctamente subida al repositorio remoto de GitHub.
2. Crear un **Pull Request (PR)** desde la rama feature hacia la rama main.
3. Revisar que el proceso de **Integración Continua (CI)** se ejecute automáticamente y finalice de forma exitosa.
4. Realizar el **merge del Pull Request** únicamente cuando todos los checks estén en estado satisfactorio.
5. Sincronizar el repositorio local con la rama main luego del merge realizado en GitHub.

Actividad 2 — Implementación de Despliegue Continuo

Se debe complementar el flujo de Integración Continua implementando un proceso de Despliegue Continuo (CD) utilizando GitHub Actions. Este proceso deberá ejecutarse automáticamente tras un push a la rama main, generando el artefacto del proyecto y publicándolo como resultado del pipeline.

Instrucciones

1. Crear un nuevo workflow de GitHub Actions para CD.
2. Configurar el workflow para que se ejecute únicamente cuando existan cambios en la rama main.
3. Asegurar que el pipeline:
 - o ejecute las pruebas automatizadas,
 - o construya el proyecto utilizando Gradle,
 - o genere el archivo ejecutable (.jar).
4. Publicar el archivo generado como **artefacto del workflow**, disponible para su descarga desde GitHub.

RESULTADOS OBTENIDOS:

- a. Realizar el informe en el formato general de informes de laboratorio.
- b. Evidencia con capturas las actividades prácticas realizadas.
- c. Anexar el código fuente en el informe.

CONCLUSIONES:

- Escribir al menos dos conclusiones.

RECOMENDACIONES:

- Escribir al menos dos recomendaciones.

FIRMAS

F:

**Nombre: Ing. Luis Castillo, Mgtr.
DOCENTE**

F:

**Nombre: Ing. Juan Fernando Galarraga, Mgtr.
COORDINADOR DE ÁREA DE CONOCIMIENTO**

F:

**Nombre: Crnl (SP) Fidel Castro de la
Cruz
JEFE DE LABORATORIO**